RDBMS → Best.

Monolithic

NoSQL

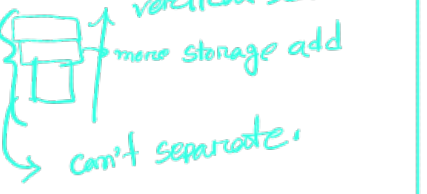
Microservice

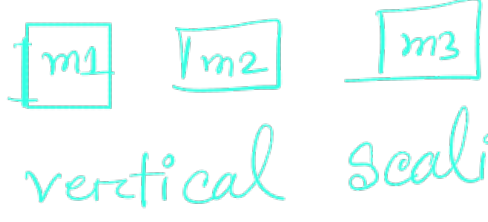
↳ by scale up.

vertical scale up

more storage add

can't separate.

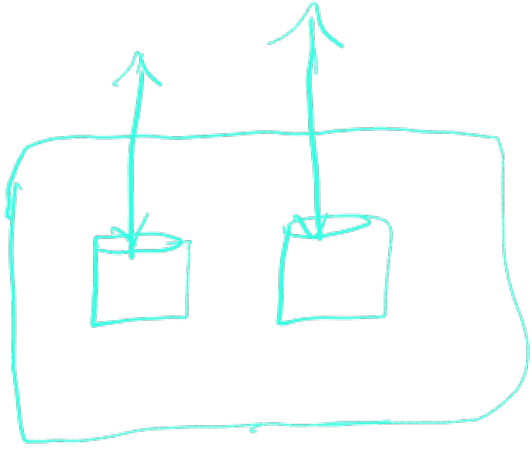




vertical scaling.

transaction property → asset property

Eventually consistent.





Natural join

where A.a = B.b (+) → all of B

↓

left join

(All of A)

Mid → 30 marks

Main Q → 3 parts (great decision qstn)

=> i) ER-D

ii) DDL

iii) A set of SQLs : ——

a)
b)
c)
⋮

→ 1st look here then go to ERD.

i) Create table

not null,
unique,
check,
PK, F.K
} constraints

Aggregate function recursive

Students

ID
Name
Dept
.....

Dept

- Name
- location
- HOD.

Dept	Total students	HOD
CSE	600	HK
EEE	400	RK

```
SELECT  count(ID) As total
FROM STUDENT S

SELECT  D.Name, Total,
        D.HOD
From    Dept D
JOIN    Student S ON
```

= S.Dept

WHERE S.

Solution:

```
Select D.Name, count(S.ID) T
   max(D.HOD)
from Student S, Depts D
Where S.Dept = D.Name
group by D.Name;
having T ≥ 500;
```

max(D.HOD) → prblm not part of grp by or func.

use min/Max → won't run (must write count(S.ID))

Where clause (✓

↳ filter Data Based on Store not on fly.

Use having. → for tot s. ≥ 500